

Installing Code::Blocks

For Managed PCs

Code::Blocks is available on the managed PC software library; IDE and MinGW (GCC compiler for Windows) installed. In the Computer lab for the module, the software will be installed, in other locations you need to use the “Software Center” (icon on desktop) and search for the program to install.

Installing Code::Blocks and MinGW to your PC/laptop:

- Go to <http://www.codeblocks.org/downloads>
- Click on ‘Download the binary release’
- Scroll to the ‘Microsoft Windows’ section
- Select codeblocks-xx.x mingw-setup.exe and click on Sourceforge.net (this should automatically start the download). NOTE: xx.xx is just whatever the current version number is.
- Click ‘Run’ (at the bottom of the screen)
- Install the Codeblocks application using the install wizard (you might need to minimise the web browser)
- After you have finished installing, click ‘Yes’ for Do you want to run Codeblocks now?
- Click ‘GNU GCC Compiler, then click ‘OK’

Quick Start

Creating a File with a C Program

During the C programming lab sessions, you will be creating Windows C program in Code::Blocks.

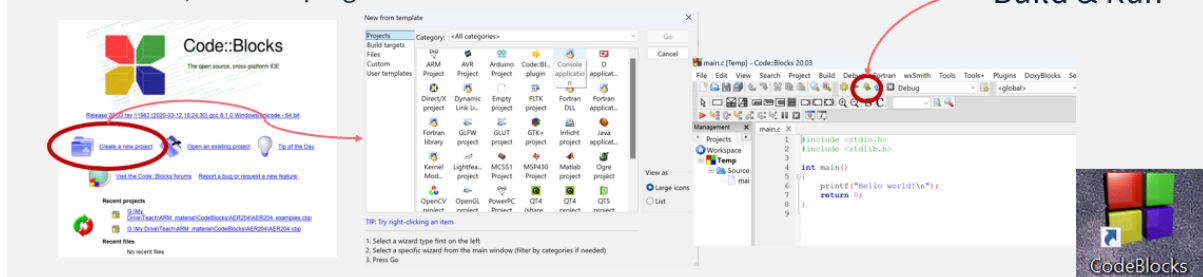
Free to download and available on managed Desktop

Create a console project; by default, a “Hello World” program is created

Set C99, debugger to start to entry, all warnings on, ...

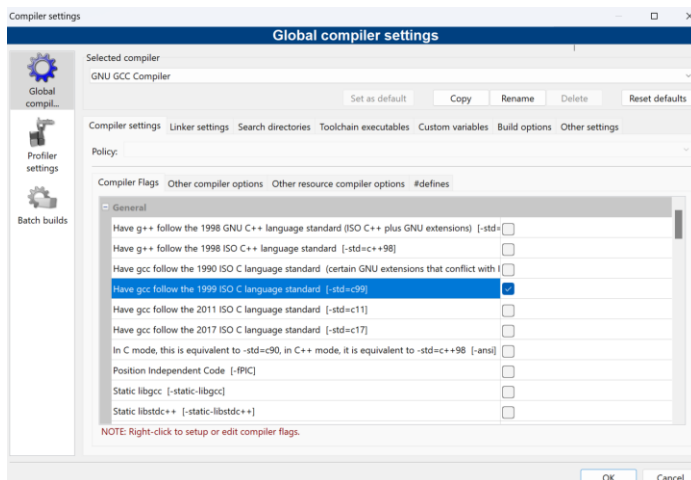
After you have written your code, you then need to:

- “Build/Run” the program



Settings for Compiler

Flags determine how the compiler behaves, these are set though the IDE GUI (you'll see this in a 'make' file later in course). Particularly important is to ensure that all warnings are enabled.

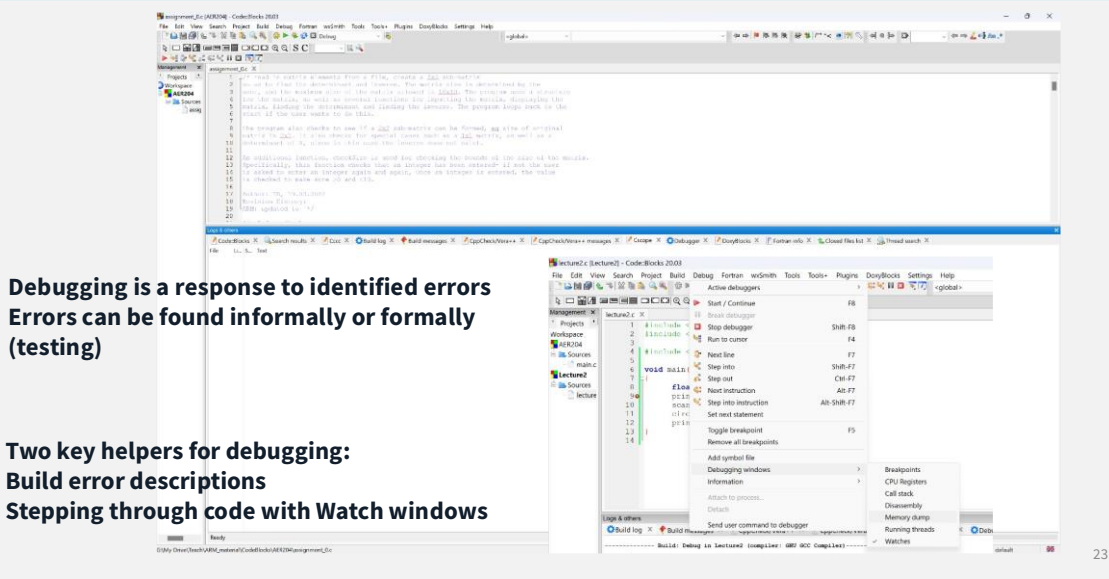


Set these flags: C99 standard, produce debug, all common warnings, no optimisation (more important for embedded system compilation).

PLEASE NOTE YOU ARE NOT EXPECTED TO FINISH EVERYTHING IN THE LAB SESSIONS. YOU NEED TO DO MORE WORK INDEPENDENTLY TO ENSURE COMPETANCE.

Debugging and Error Messages

Code::Blocks Debugging & Compiler Errors



Debugging is a response to identified errors
Errors can be found informally or formally (testing)

Two key helpers for debugging:
Build error descriptions
Stepping through code with Watch windows

Debugging in Code::Blocks helps you find and fix errors in your code. The process involves setting breakpoints, stepping through your code line by line, and inspecting variable values as the program runs.

1. Setting Up Your Project for Debugging

Before you can debug, you must have a project. If you're working with a single C file, create a new "Console application" project and add your file to it.

- Go to **File > New > Project...**
- Select **Console application**, and click **Go**.
- Choose **C** as the language.
- Give your project a name and save it.
- Add your C source file to the project.

2. Setting Breakpoints

A **breakpoint** is a marker you place on a line of code where you want the program execution to pause. This allows you to inspect the program's state at that specific point.

- Open your C source file in the editor.
- Click in the gray margin to the left of the line number where you want to set a breakpoint. A red circle will appear, indicating the breakpoint is active.

3. Starting the Debugger

Once you have your breakpoints set, you can start the debugging session.

- Go to **Debug > Start/Continue**.
- Alternatively, you can press the **F9** key.

The program will run until it hits the first breakpoint, and the line will be highlighted.

4. Stepping Through Code

While the program is paused at a breakpoint, you can use these commands to control its execution:

- **F7 (Next line):** Executes the current line of code and moves to the next one. This is the most common command for stepping through your program.
- **F8 (Next instruction):** Similar to F7 but is more detailed, sometimes stepping into assembly instructions. Use F7 for most cases.
- **F4 (Next line of code - skip functions):** Skips over function calls, executing the entire function and stopping on the line after it returns. Use this to avoid stepping through functions you know are correct.

5. Watching Variables

One of the most powerful features of a debugger is the ability to inspect the values of variables in real-time.

- Go to **Debug > Debugging Windows > Watches**.
- In the "Watches" window that appears, you can see the values of variables as you step through the code.
- You can add a variable to the watch list by right-clicking it in the editor and selecting **"Add watch"** or by manually typing its name in the "Watches" window.

Common C Error Messages and How to Interpret Them

`expected ';' before 'token'`

- **Meaning:** The compiler expects a semicolon at a certain point but found something else instead.
- **Common Causes:**
 - Forgetting a semicolon at the end of a statement.
 - This error can sometimes appear on the line *after* the actual error, as the compiler doesn't realize the semicolon is missing until it sees the next line of code.

`conflicting types for 'variable_name'`

- **Meaning:** You have declared the same variable or function with two different data types.
- **Common Causes:**
 - Declaring a global variable with one type and then a local variable in a function with the same name and a different type.
 - This can also happen with functions if you declare a function prototype with one return type but define the function with another.

'variable_name' undeclared (first use in this function)

- **Meaning:** The compiler has no record of the variable you are trying to use.
- **Common Causes:**
 - Forgetting to declare the variable before using it.
 - Misspelling the variable name. C is case-sensitive, so `myVar` and `myvar` are different.

*warning: format '%d' expects type 'int *', but argument '2' has type 'int'*

- **Meaning:** You are using the wrong format specifier in a function like `scanf` or `printf`. The message above specifically means you passed an `int` instead of a pointer to an `int` to `scanf`.
- **Common Causes:**
 - Using `scanf("%d", num);` instead of the correct `scanf("%d", &num);`. The `&` operator is crucial for `scanf` to pass the address of the variable so it can store the input value.
 - Using the wrong format specifier, e.g., `%f` for a double instead of `%lf`.

undefined reference to 'function_name'

- **Meaning:** The linker can't find the definition for a function that you have called. This usually happens when you've declared a function but haven't included its definition.
- **Common Causes:**
 - Forgetting to include a library header file (e.g., `<string.h>` for `strlen()`).
 - Misspelling the function name.
 - Trying to use a function that is part of a library you haven't linked to (less common in simple C programs).

